

Trabalho de Segurança da Computacional - CIC/UnB

Nome: Samuel Carvalho dos Santos – 211010388

Professor: João Jose Costa Gondim

2024.1

Relatório

Introdução.

Este trabalho de Segurança computacional - CIC, do semestre de 2024.1; tem por finalidade e realização, a Implementação do código AES (Advanced Encryption Standart) de 128 bits, do código do Modo CTR (Counter) e do código do Modo GCM (Galois/Counter Mode); contendo também testes sendo realizados com arquivos de texto TXT e imagem PNG. Dentre as diferentes metodologias de criptografia implementadas manualmente a fim de, aprender os processos de cada método; também foram implementadas da Parte II do trabalho, a Etapa 1, Geração de chaves (p e q primos com no mínimo de 1024 bits) e Cifração/decifração assimétrica RSA usando OAEP; sendo elas testadas através de texto em código.

1.AES.

O AES, Padrão de Criptografia Avançada (Advanced Encryption Standart); é um subconjunto da cifra de bloco Rijndael baseada em um princípio de design conhecida como, rede de substituição-permutação, desenvolvida por dois criptógrafos belgas, Vincent Rijmen e Joan Daemen. Para o AES, o NIST selecionou três membros da família Rijndael, cada um com um tamanho de bloco de 128 bits, mas três comprimentos de chave diferentes: 128 (A abordada no trabalho), 192 e 256 bits. O algoritmo opera em um arranjo de bytes de ordem principal de linha e coluna 4x4, tendo seus cálculos realizados em um campo finito.

O código AES se encontra na versão simplificada no AES de 128 bits, incluindo funções básicas para seu funcionamento de criptografia e descryptografia codificadas em Python. O tamanho da chave especifica o número de rodadas de transformação que convertem a entrada, texto simples e na saída final, texto cifrado; para o caso do AES 128, 10 rodadas é o especificado. Cada rodada se consiste em várias etapas do processamento, incluindo a que depende da própria chave de criptografia; para o caso de rodadas reversas é usado para modificar o texto cifrado para o original utilizando a mesma chave de criptografia.

1.0 Descrição do algoritmo padrão.

1. KeyExpansion: as chaves das rodadas são derivadas da chave de cifragem, usando o planejamento de chave AES, requerendo uma rodada de bloco, de chave de 128 bits separada para cada rodada mais um.
2. AddRoundKey: Adição da chave da rodada inicial, cada byte do estado é combinado com um byte da chave da rodada usando operações bit a bit xor.
3. Rodadas: Em cada rodada são efetuadas estas etapas.
 - 3a. SubBytes: Etapa de substituição não linear onde cada byte é substituído por outro de acordo com a tabela de pesquisa. Utilizando uma caixa de substituição de 8 bits, acarreta fornecer a não linearidade na cifra, a caixa de substituição é utilizada, é derivada do inverso multiplicativo sobre corpo finito. A fim de evitar ataques baseados em propriedades algébricas simples ela é construída combinando a função inversa com uma transformação afim invertível.
 - 3b. ShiftRows: Etapa de transposição em que as três últimas linhas do estado, são deslocadas ciclicamente em um certo número de etapas. Para o AES, a primeira linha permanece inalterada, cada byte a segunda é deslocado um para a esquerda. Assim como a terceira e a quarta linha são deslocadas; cada coluna de saída de estado da etapa ShiftRows é composta de bytes de cada coluna de entrada de estado, evitando que as colunas sejam criptografadas de forma independente, caso em que o AES se degradaria em quatro cifras independentes.
 - 3c. MixColumns: Operação de mistura linear que opera nas colunas do estado, combinando os quatro bytes em cada coluna. Leva quatro bytes como entrada e produz quatro bytes, onde cada byte de entrada afeta todos os quatro bytes de saída; juntamente com a ShiftRows, o MixColumns fornece difusão na cifra. Durante esta operação cada coluna é transformada utilizando uma matriz fixa, a matriz multiplicada à esquerda.

3d. AddRoundKey: A subchave é combinada com o estado; uma subchave é derivada da chave principal usando o planejamento AES e cada subchave tem o mesmo tamanho que o estado.

4. Rodada final (fazendo 10, 12 ou 14 rodadas no total):

4a. SubBytes

4b. ShiftRows

4c. AddRoundKey

1.1 Implementação AES.

Neste tópico é apresentada a Implementação do código AES realizada, assim como, descrevendo-a. O algoritmo “Aes_128” foi implementado em uma classe em um arquivo próprio “Implementacao_AES.py”.

. Inicialização da Classe: A classe “Aes_128” é inicializada com duas matrizes (sbox e a rev_sbox), tabelas de substituição utilizadas para criptografar e descriptografar. A “sbox” é uma matriz de 16x16 para substituição não linear durante a criptografia, tendo cada byte substituindo por um byte correspondente na “sbox”. A “rev_sbox”, por sua vez, é semelhante à “sbox”, porém para a descriptografia. Sendo assim, o inverso da “sbox”.

. Visualizar (visualizar e visualizar_reverse): São utilizadas para efetuar a substituição de bytes usando as tabelas da inicialização. Elas pegam um byte como entrada e divide em dois para indexar a tabela respectiva, retornando o valor substituído. Se equivale à “SubBytes” do algoritmo padrão.

. Rotacionar à Esquerda (Rotacionar_Esq): Ela rotaciona uma lista à esquerda por um número ‘n’ de vezes. Sendo utilizada na “ShiftRows” do AES. Exemplo de uso: Rotacionar_Esq([1, 2, 3, 4], 1) retorna [2, 3, 4, 1].

. Quebrar em Grades de 16 (break_in_grids_of_16): Faz a quebra do dado em blocos de 16 bytes (128 bits); cada bloco é então organizado em uma matriz 4x4, estrutura padrão dos blocos de dados no AES. Processo: Tendo uma entrada de

um array de bytes, cada bloco de 16 bytes é organizado e uma matriz 4x4, onde célula da matriz contém 1 byte.

. **Expansão da Chave (Expansao_Chave):** Ela gera várias subchaves (Round Keys) a partir da chave inicial de 128 bits; essas subchaves são usadas em cada rodada de encriptação ou deciptação.

-**Rcon:** é uma lista constante usada na expansão da chave usada para garantir que cada rodada use uma chave unica.

-**Processo de Expansão:**

. **Divisão da chave inicial:** A chave é dividida em várias colunas de 4 bytes.

.**temp:** Rotaciona uma palavra a esquerda.

.**temp:** Substitui cada byte da palavra usando a matriz 'sbox'.

.**temp[0]:** O resultado da "temp" é combinado com o "rcon" e com um dos dados anteriores para formar o novo dado.

-**Geração de Chaves para Rodadas:** As subchaves são geradas a partir das operações mencionadas anteriormente (Processo de Expansão) e são usados em cada rodada do AES para realizar o "AddRoundKey" (AdicionarSubKey).

.**AddicionarSubKey, Visualizar, Rotacionar_Esq e misturar_colunas:** São operações fundamentais do processo de criptografiaAES, tendo:

-AddicionarSubKey: Realiza uma operação XOR entre o bloco de dados e a subchave para a que foi rotacionada.

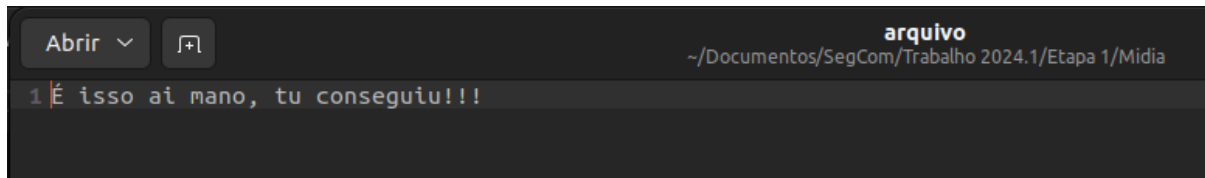
-Visualizar: Utiliza a 'sbox' para substituir cada byte do bloco.

-Rotacionar_Esq: Rotaciona as linhas da matriz do bloco de dados para a esquerda.

-misturar_colunas: Realiza a mistura linear dos dados da coluna da matriz.

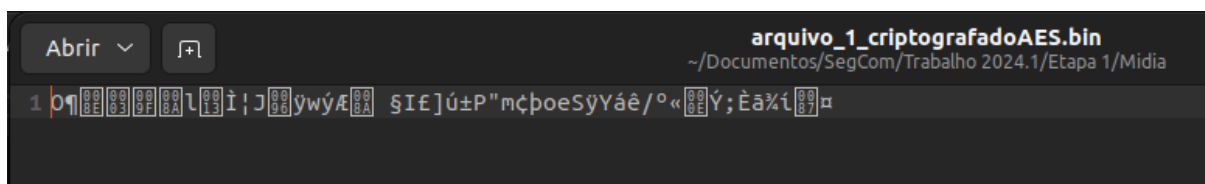
.multiplicar_x: Funções auxiliares usadas na expansão de chave para calcular o próximo valor de 'rcon' na série.

Arquivo Original.



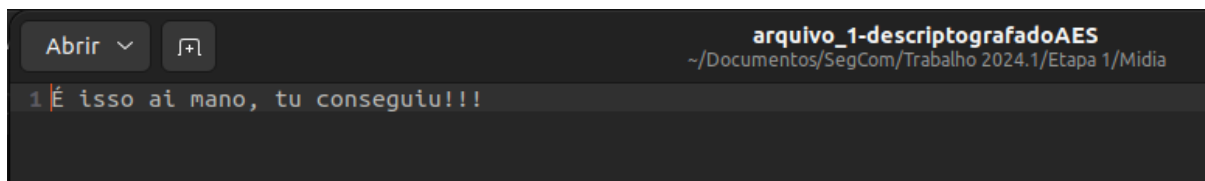
A screenshot of a text editor window. The title bar shows 'arquivo' and the path '~/Documentos/SegCom/Trabalho 2024.1/Etapa 1/Midia'. The editor contains a single line of text: '1 É isso aí mano, tu conseguiu!!!'.

Arquivo Criptografado pelo Algoritmo AES 128.



A screenshot of a text editor window. The title bar shows 'arquivo_1_criptografadoAES.bin' and the path '~/Documentos/SegCom/Trabalho 2024.1/Etapa 1/Midia'. The editor contains a single line of text that is a mix of readable characters and hex codes, representing the encrypted version of the original file.

Arquivo Descriptografado pelo Algoritmo AES 128.



A screenshot of a text editor window. The title bar shows 'arquivo_1-descriptografadoAES' and the path '~/Documentos/SegCom/Trabalho 2024.1/Etapa 1/Midia'. The editor contains a single line of text: '1 É isso aí mano, tu conseguiu!!!', which matches the original file content.

1.2 Implementação CTR.

O modo Counter (CTR) é um modo de operação de cifra de bloco típico usando algoritmo de cifra de bloco. Fornece ao AES capacidade de processamento de comprimentos além dos de 128/192/256 bits. A criptografia possui um modo de contador denominado por CTR, atendendo pelos nomes de ICM e SIC. Na criptografia AES, se possui um Vetor de Inicialização ou 'IV'; geralmente é uma entrada aleatória de 128 bits. O IV possui 2 componentes no Modo CTR, o Randomizado regular, que está presente nos primeiros 8 bytes e há um contador nos 8 bytes finais. Este contador é um índice 0 que indica quantos blocos de 128 bits dos dados criptografados se possui acesso. Quando 512 bits de dados (64 bytes) estão sendo criptografados, a posição inicial em 0 bytes nos dados teria um contador de 0; em 16 bytes, um contador de 1; e em 32 bytes um contador até 2, até ter esgotado todas as suas informações; essa criptografia pode ser pesquisada através das informações, ao contrário da criptografia AES padrão. Não é necessário descriptografar todos os bytes para obter algumas informações no meio.

O código apresentado neste trabalho foi realizado aproveitando a classe 'Aes_128' já implementada, utilizando a linguagem Python.

. Importações e Classe AES_CTR: A classe AES_CTR é inicializada com a importação e instanciação da classe Aes_128 para utilização de suas funções.

.incrementar_contador: O modo CTR utiliza um contador que é criptografado a cada bloco de dados; essa função incrementa o valor do contador de 16 bytes. O contador é tratado como um inteiro grande ('int.from_bytes'), incrementado em 1, e depois convertido de volta para uma sequência de bytes ('to_bytes').

.ctr_encrypt: O método ctr_encrypt recebe a chave, o dado a ser criptografado e o número de rodadas de criptografia. Se o dado for uma string, ele é convertido para bytes (dado.encode()). O dado é dividido em blocos de 16 bytes; isso é necessário para compatibilidade do AES128, cada bloco é armazenado na lista 'blocos'. O contador é iniciado com 16 bytes zerados, este contador será criptografado para gerar a Keystream.

Para cada bloco de dados, o valor atual do contador é criptografado usando AES. O resultado da criptografia do contador é o keystream. O bloco de dados é combinado com o keystream usando a operação XOR. O bloco criptografado resultante é adicionado ao dado criptografado final ('dado_encryptado').

Após cada bloco, o contador é incrementado para garantir que cada bloco de dados seja criptografado com m keystream diferente. O dado criptografado completo é retornado.

.ctr_decrypt: No Modo CTR, a descriptografia é idêntica à criptografia; pois o dado criptografado pode ser obtido usando a mesma operação XOR com o keystream que foi usado para criptografar; desta forma a função 'ctr_decrypt' apenas chama a 'ctr_encrypt' com os mesmos parâmetros.

Arquivo Original.

. Importação e Definição da Classe AES_GCM: O código importa a implementação do 'Aes_128'. A classe está encapsulada e se inicia instanciando a classe importada 'Aes_128' que será utilizada para a criptografia.

.ghash: Ela calcula a função GHASH, que é usada no Modo GCM para gerar uma tag de autenticação. A função 'galois_mult' realiza uma multiplicação no campo de Galois sobre 128 bits. O processo de GHASH se consiste em inicialmente, h ser convertido de bytes para um inteiro. Tendo para cada bloco de dados, o bloco sendo convertido para um inteiro e combinado com 'y' usando XOR; a multiplicação de Galois é realizada entre 'y' e 'h'; o resultado é mantido dentro do espaço de 128 bits e o valor 'y' é convertido de volta para bytes.

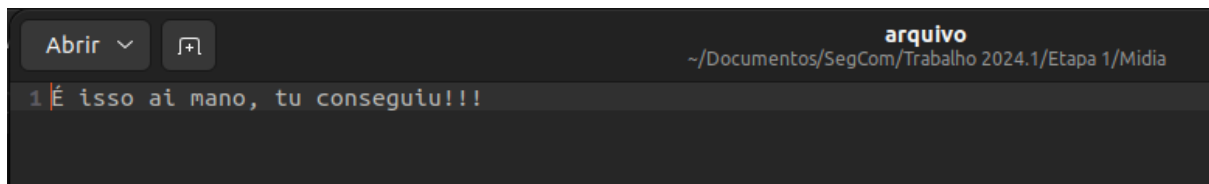
.divide_em_blocos: Divide os dados de entrada em blocos de 16 bytes, tamanho usado no AES 128.

.gctr: Esta função implementa o modo de operação GCTR (Galois Counter Mode), que é usado tanto para criptografia quanto para geração de keystreams. Ela possui como processo, a divisão de blocos em 16 bytes, sendo que para cada bloco; o contador é criptografado para gerar o keystream; o bloco é combinado com o keystream usando XOR; o contador é incrementado para o proximo bloco e por fim, o dado criptografado é retornado.

.encrypt: A chave de autenticação 'h' é gerada criptografando um bloco de zeros com a chave AES; o contador inicial 'j0' é gerado concatenando o 'nonce' com o valor '1'; o texto é criptografado usando GCTR com j0 como o contador inicial. Para o cálculo da tag o GHASH é calculado sobre o AAD, o texto cifrado, e os tamanhos do AAD e do texto e a tag é gerada criptografando 's' com 'gctr'. Por fim a função retorna o texto cifrado e a tag de autenticação.

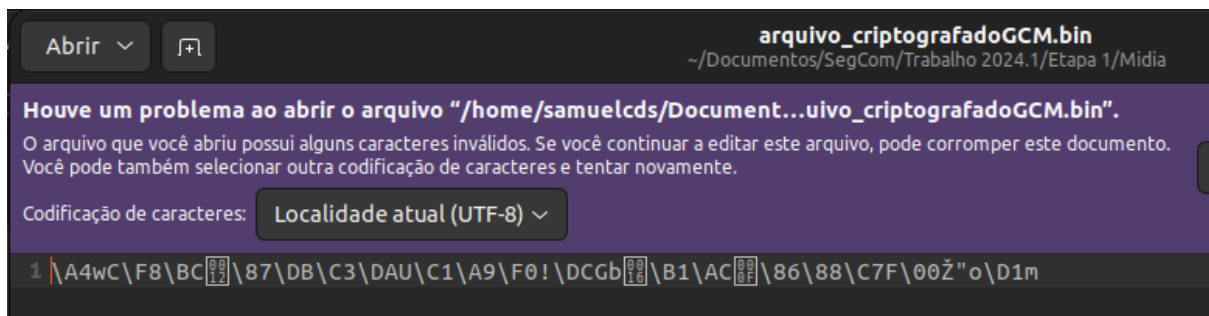
.decrypt: O texto cifrado é descriptografado usando 'gctr' com o contador inicial 'j0'. O cálculo da tag é efetuado com O 'ghash' é calculado sobre o 'aad', o texto cifrado, e os tamanhos e a tag sendo gerada da mesma forma que na criptografia. A tag gerada é comparada com a tag fornecida, se as tags não coincidirem, uma exceção é levantada, indicando a falha. Por fim o texto descriptografado é retornado.

Arquivo Original



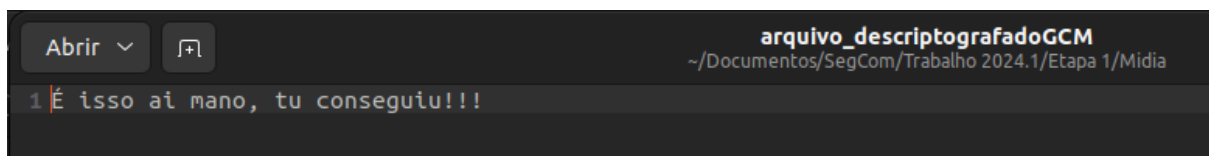
A screenshot of a text editor window titled "arquivo". The path shown is "~/Documentos/SegCom/Trabalho 2024.1/Etapa 1/Midia". The text content is "1 É isso aí mano, tu conseguiu!!!".

Arquivo Criptografado pelo Algoritmo GCM.



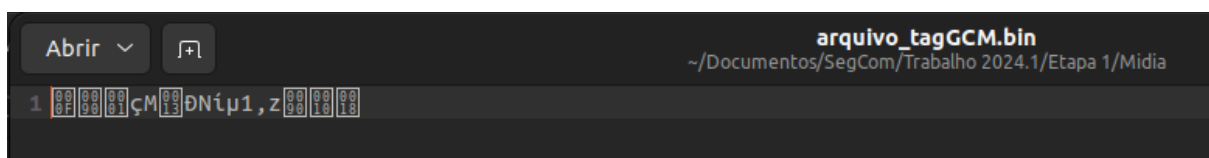
A screenshot of a text editor window titled "arquivo_criptografadoGCM.bin". The path shown is "~/Documentos/SegCom/Trabalho 2024.1/Etapa 1/Midia". A purple error message is displayed: "Houve um problema ao abrir o arquivo "/home/samuelcds/Document...uivo_criptografadoGCM.bin". O arquivo que você abriu possui alguns caracteres inválidos. Se você continuar a editar este arquivo, pode corromper este documento. Você pode também selecionar outra codificação de caracteres e tentar novamente." Below the message, it says "Codificação de caracteres: Localidade atual (UTF-8)". The text content is a garbled string: "1 \A4wC\F8\BC[invalid]\87\DB\C3\DAU\C1\A9\F0!\DCGb[invalid]\B1\AC[invalid]\86\88\C7F\00Ž"o\D1m".

Arquivo Descriptografado com o Algoritmo GCM.



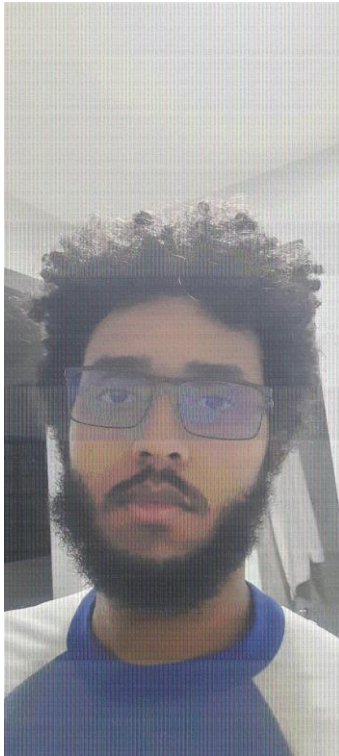
A screenshot of a text editor window titled "arquivo_descriptografadoGCM". The path shown is "~/Documentos/SegCom/Trabalho 2024.1/Etapa 1/Midia". The text content is "1 É isso aí mano, tu conseguiu!!!", which matches the original file.

Arquivo TAG gerado pelo Algoritmo GCM.

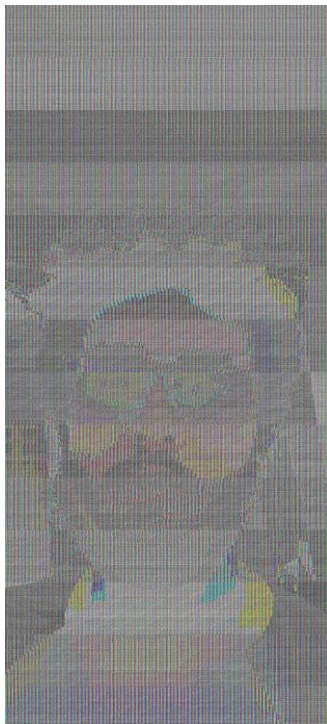


A screenshot of a text editor window titled "arquivo_tagGCM.bin". The path shown is "~/Documentos/SegCom/Trabalho 2024.1/Etapa 1/Midia". The text content is a hexadecimal string: "1 [hexadecimal characters]".

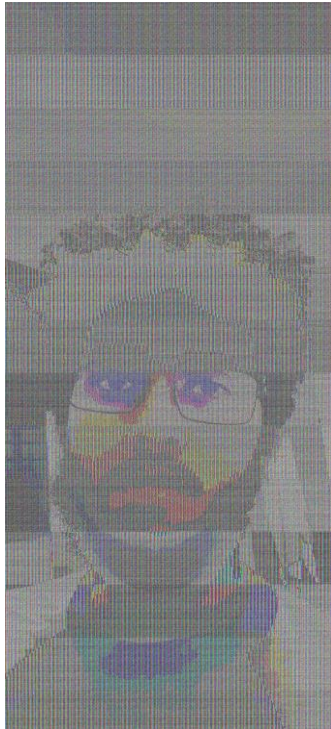
1.4 Testes com selfies.



1 Rodada.



5 Rodadas.



9 Rodadas.



13 Rodadas.

2.RSA

O RSA é um esquema matemático que visa apresentar a autenticidade de mensagens ou documentos digitais; essas assinaturas digitais normalmente são usadas para distribuição de software, transações financeiras etc. Para ser considerada válida, uma assinatura digital deve apresentar três principais características:

- **Autenticação:** Uma assinatura digital válida fornece um motivo para o destinatário acreditar que a mensagem foi criada por um remetente conhecido.

- **Não-repúdio:** O remetente não pode negar o envio da mensagem.

- **Integridade:** Que a mensagem não foi alterada em trânsito.

Um esquema de assinatura digital normalmente consiste em 3 algoritmos: Um algoritmo de geração de chaves que gera uma chave privada de maneira uniforme e aleatória e uma chave pública correspondente. Um algoritmo de assinatura que, dada uma mensagem e uma chave privada, produz uma assinatura. Um algoritmo de verificação de assinatura que, dada a mensagem, chave pública e assinatura, aceita ou rejeita a reivindicação de autenticidade da mensagem.

2.1 Gerador de Chaves.

.mod_inverso: Tem como objetivo encontrar o inverso modular de constante mod $\phi(n)$, onde $\phi(n)$ é a função totiente de Euler do número n . O algoritmo de Euclides Estendido é usado para resolver equações do tipo $ax + by = \text{máximo divisor comum}$. A função `mod_inverso` ajusta essa abordagem para calcular o inverso modular. E a função tem por funcionamento a iteração, subtraindo múltiplos de y_3 de x_3 até que y_3 seja zero; sendo que Se x_3 não for igual a 1, a constante e $\phi(n)$ não são coprimos, e o inverso modular não existe e o valor final de $x_2 \% \phi_n$ é o inverso modular que estamos procurando.

.eh_primo: Verifica se um número é primo; primeiramente descartando números menores que 2; em seguida, trata 2 e 3 como casos especiais, pois são primos; números divisíveis por 2 ou 3 são imediatamente descartados; para números maiores, um laço testa divisores possíveis até a raiz quadrada do número, incrementando de 6 em 6 para verificar possíveis divisores e se nenhum for encontrado, o número é primo.

.miller_rabin: Realiza um teste de primariedade probabilístico chamado Miller-Rabin, que é usado para verificar se um número é primo com alta probabilidade. Tendo por funcionamento O número ' n ' sendo testado diretamente se for 2 ou 3 (primos) e se ' n ' for par ou igual a 1, é imediatamente descartado; escrevendo ' $n-1$ ' na forma ' $2^r * \text{mod}$ ', isolando as potências de 2; sendo que, para cada um dos k testes, um número aleatório a é escolhido, e o teste de Miller-Rabin é aplicado: Se ' $a^{\text{mod}} \% n$ ' for 1 ou ' n '

1', o teste é considerado um sucesso para esse valor, caso contrário, a função tenta iterar, elevando ao quadrado e verificando ' $a^{\text{mod } n}$ ', se nenhum valor satisfizer a condição, n é considerado composto. Passando pelos testes, é considerado primo.

.gerar_primo: Ela gera um número primo grande com o número especificado de bits. Funciona tendo um número aleatório 'num' é gerado com o número de bits especificado; o número logo é ajustado para ser ímpar e garantir que tenha o número correto de bits; o teste de primariedade de Miller-Rabin é aplicado para verificar se 'num' é primo e quando ele é encontrado, é retornado.

.gerar_chave_rsa: Ela gera um par de chaves RSA (pública e privada). Com dois números primos grandes (primeiro e segundo) sendo gerados usando 'gerar_primo' e se os dois números forem iguais, um novo segundo é gerado; o módulo n é calculado como o produto dos dois primos. A função totiente de Euler, $\phi(n)$, é calculada como $(\text{primeiro} - 1) * (\text{segundo} - 1)$. O expoente público (constante) é definido como 65537, um valor comumente usado; o inverso modular de constante mod $\phi(n)$ é calculado para obter o expoente privado e as chaves pública e privada são retornadas. A chave pública é $(\text{constante}, n)$, e a chave privada é (mod, n) .

2.2. Implementação do RSA usando OEAP.

O OAEP garante que a mensagem tenha um formato específico antes de ser cifrada, o que ajuda a evitar certos tipos de ataques criptográficos.

.xor_bytes: Realiza uma operação XOR byte a byte entre duas sequências de bytes. Ela combina os bytes correspondentes das duas sequências; a operação XOR (^) é aplicada a cada par de bytes e o resultado é convertido de volta para uma sequência de bytes.

.generate_mask: Gera uma máscara criptográfica de um comprimento especificado usando uma função hash (importada). Para o funcionamento, o seed (semente) é combinado com um contador (C), convertido para bytes; a função hash (por padrão, SHA-256) é aplicada ao seed + C para produzir uma sequência de bytes, este processo é repetido, incrementando o contador, até que o 'result' tenha o comprimento desejado. Por fim, máscara gerada é então usada para "ofuscar" os dados na etapa de padding.

.oeap_pad: Aplica o padding OAEP a uma mensagem antes de cifrá-la. Funciona com `h_len` sendo o tamanho do digest gerado pela função hash (SHA-256 gera 32 bytes). Calcula-se o tamanho da string de preenchimento (PaddingString, `ps`), que é usada para preencher o bloco até o tamanho necessário; a `db` (data block) é composto pelo hash de uma string vazia, seguido pela `ps`, um byte 0x01 como delimitador, e finalmente a

mensagem; a seed é uma sequência de bytes aleatória de tamanho `h_len`; uma máscara `db_mask` é gerada a partir da seed, e o `db` é mascarado com esta máscara usando a função `xor_bytes` e uma máscara `seed_mask` é gerada a partir do `masked_db`, e a seed é mascarada com esta máscara. O resultado final é o byte `0x00` seguido do `masked_seed` e do `masked_db`, formando o bloco codificado.

.oeap_unpad: Sendo o processo inverso da '`oeap_pad`'. Verificando se o comprimento da mensagem preenchida é igual ao tamanho do bloco esperado. Extraíndo as partes `masked_seed` e `masked_db` da mensagem preenchida. Gerando a `seed_mask` a partir do `masked_db`, e a usa para recuperar a seed. Gerando a `db_mask` a partir da seed, e a usa para recuperar o `db`. Verificando se o início de `db` corresponde ao hash esperado de uma string vazia (`l_hash`); e localizando o byte delimitador `0x01` e recupera a mensagem original, ignorando a Padding String.

Bibliografia

. https://pt.wikipedia.org/wiki/Advanced_Encryption_Standard (AES).

. <https://medium.com/wearesinch/building-aes-128-from-the-ground-up-with-python-8122af44ebf9> (Exemplo de Implementação AES em Python).

. <https://github.com/rafael2903/AES-128-cipher/tree/main> (Exemplo de AES em Python).

. https://xilinx.github.io/Vitis_Libraries/security/2020.1/guide_L1/internals/ctr.html#:~:text=Overview,bits%20before%20encryption%20or%20decryption. (Base teorica CTR).

. <https://www.javatpoint.com/aes-ctr-python> (Base teórica CTR).

. <https://github.com/openssl/openssl/blob/master/crypto/modes/ctr128.c>
(Exemplo de implementação CTR).

. https://github.com/Legrandin/pycryptodome/blob/master/src/raw_ctr.c
(Exemplo de implementação CTR).

. https://github.com/Legrandin/pycryptodome/blob/master/lib/Crypto/Cipher/_mode_ctr.py (Exemplo de implementação CTR).

. <https://nvlpubs.nist.gov/nistpubs/legacy/sp/nistspecialpublication800-38d.pdf> (Base teórica GCM).

- . https://en.wikipedia.org/wiki/Galois/Counter_Mode (Base teórica GCM).
- . <https://github.com/openssl/openssl/blob/master/crypto/modes/gcm128.c> (Exemplo de implementação GCM).
- . <https://www.youtube.com/watch?v=r7qNYQXAq9g> (Exemplo de implementação GCM).
- . <https://akiradev.netlify.app/posts/manipulando-imagens-python/> (Consulta para Manipular imagens).
- . <https://br.matplotlib.net/stable/tutorials/introductory/images.html> (Consulta para Manipular imagens).
- . <https://medium.com/@jonathas.mpf/assinatura-digital-com-python-d03df25116fb> (Base teórica RSA).
- . <https://blog.zapsign.com.br/criptografia-rsa/> (Base teórica RSA).
- . [https://pt.wikipedia.org/wiki/RSA_\(sistema_criptogr%C3%A1fico\)](https://pt.wikipedia.org/wiki/RSA_(sistema_criptogr%C3%A1fico)) (Base teórica RSA e Exemplo).
- . <https://github.com/tarcisio-marinho/RSA> (Exemplo de código).
- . https://pt.wikipedia.org/wiki/Algoritmo_de_Euclides_estendido
- . <https://dl.ebooksworld.ir/books/Introduction.to.Algorithms.4th.Leiserson.Stein.Rivest.Cormen.MIT.Press.9780262046305.EBooksWorld.ir.pdf> (Inverso modular).
- . <https://incolumitas.com/2018/08/12/finding-large-prime-numbers-and-rsa-miller-rabin-test/> (Miller).
- . <https://medium.com/@ntnprdhmm/how-to-generate-big-prime-numbers-miller-rabin-49e6e6af32fb> (Miller).
- . <https://github.com/felipenrocha/RSA-OAEP> (Exemplo de código).